

面试知识库 — 04-高频追问与源码级应答库

定位: 技术深挖防御层 + 简历每个关键词的深度展开 **核心价值:** 面试官追问"具体怎么实现的"时, 能立刻给出架构级回答+关键代码片段 **使用方式:** 技术二面/三面/总监面时按模块检索

一、简历独有关键词深度拆解

1.1 Redis+Lua高并发计数体系

简历原文: "基于 Redis + Lua 脚本重构高并发计数体系: 摒弃直接更新数据库的方案, 利用 Lua 脚本保证计数原子性, 结合本地缓存与滑动时间窗算法在内存层拦截重复请求, 最终通过定时任务异步批量落库"

面试官可能追问的方向:

- 为什么不用Redis INCR/DECR而要用Lua?
- 滑动时间窗怎么实现的?
- 异步批量落库怎么保证不丢数据?
- 和项目二的点赞系统有什么区别?

完整技术方案描述:

这个计数体系是VRS项目中为视频播放量、点赞数、评论数等高频计数场景设计的。原始方案是每次用户操作直接UPDATE数据库的count字段——问题在于: 一个热门视频被1000人同时点赞时, 数据库要执行1000次UPDATE, 行锁竞争严重, CPU飙升到100%。

三层架构设计:

```
Layer1: 本地Caffeine缓存(滑动时间窗口去重)
    ↓ 命中则直接返回,拦截~70%重复请求
Layer2: Redis Hash + Lua脚本(原子计数)
    ↓ 写入成功立即返回,异步通知Layer3
Layer3: 定时任务批量落库(MySQL/PostgreSQL)
    ↓ 每30秒一批,saveBatch批量写入
```

Layer1 滑动时间窗去重原理: 维护一个ConcurrentMap<userId+actionType+targetId, timestamp>, 窗口期60秒内同一用户对同一目标的同一操作只计一次。用ScheduledExecutorService每10秒清理过期条目。关键点是用computeIfAbsent 保证线程安全——如果key已存在说明在窗口期内, 直接返回"已计数"; 不存在则放入当前时间戳并放行到Layer2。这一层拦截了约70%的重复点击(用户快速双击、恶意刷新)。

Layer2 Lua脚本核心逻辑:

```
-- 核心伪码结构(非完整代码)
local key = KEYS[1]          -- "video:stats:{videoId}"
local field = ARGV[1]        -- "play_count" / "like_count"
local action = ARGV[2]       -- "incr" or "decr"
local delta = tonumber(ARGV[3]) -- 通常为1

local current = redis.call('HGET', key, field) or "0"
```

```
if action == "incr" then
    redis.call('HINCRBY', key, field, delta)
else
    local newVal = tonumber(current) - delta
    if newVal >= 0 then
        redis.call('HSET', key, field, tostring(newVal))
    else
        return -1 -- 不允许负数
    end
end
return tonumber(redis.call('HGET', key, field))
```

为什么用Lua而不是MULTI/EXEC事务?

对比维度	MULTI/EXEC	Lua脚本
原子性	☑ 但需客户端发送多条命令	☑ 服务端单次执行
条件判断	✗ 只能顺序执行	☑ 支持if/else
网络往返	多次RTT	单次RTT(+SHA缓存后更短)
复杂度	低	中(但可复用)

Layer3 定时批量落库: Spring @scheduled(fixedDelay = 30000) 触发，流程：①SCAN所有 video:stats:* Key; ②HGETALL取出所有字段值；③与DB当前值做DIFF计算delta；④生成批量UPDATE SQL；⑤ jdbcTemplate.batchUpdate() 执行。失败重试3次指数退避。最坏情况下Redis宕机丢失的数据量 = 30秒窗口内的增量 ≈ 几百到几千（取决于流量），属于可接受范围。

量化效果:

指标	重构前(DB直写)	重构后(三层架构)
数据库写QPS	~2000(峰值)	~50(批量合并后)
DB写操作减少率	基准	97.5%
计数响应延迟	20-50ms	<1ms(Redis)
重复请求拦截	无	~70%(时间窗)
数据一致性	强一致	最终一致(≤30s延迟)

1.2 ORM层SQL拦截器与全局逻辑隔离

简历原文: "通过 ORM 层的 SQL 拦截器实现未审核内容的全局逻辑隔离"

面试官可能追问:

- 为什么不在Service层加if判断?
- 拦截器的性能开销?
- 怎么避免拦截到管理后台的查询?

技术方案:

这是VRS项目的安全基线设计——确保任何面向前台用户的查询都不会返回APPROVED状态以外的视频。如果在每个Service方法里手动加 `WHERE status='APPROVED'`，容易遗漏且违反DRY原则。

MyBatis Interceptor实现要点:

```
@Intercepts({
    @Signature(type=StatementHandler.class, method="prepare",
        args={Connection.class, Integer.class})
})
public class AuditStatusInterceptor implements Interceptor {

    @Override
    public Object intercept(Invocation invocation) {
        StatementHandler handler = (StatementHandler) invocation.getTarget();
        BoundSql boundSql = handler.getBoundSql();
        String originalSql = boundSql.getSql().toLowerCase();

        if (isSelectOnAuditedTable(originalSql) && !isAdminContext()) {
            String modifiedSql = appendAuditCondition(originalSql);
            setFieldValue(boundSql, "sql", modifiedSql);
        }
        return invocation.proceed();
    }
}
```

三个关键设计决策:

决策1: 拦截粒度选择StatementHandler而非Executor

- Executor级别拦截范围太广，会影响所有SQL
- StatementHandler的prepare阶段刚好在SQL发送给DB之前，可以精确改写
- 配合 `@signature` 注解只拦截prepare方法

决策2: 如何区分前台和管理后台查询? 用ThreadLocal传递上下文信息:

```
// 在Controller层或Filter层设置
AuditContext.setRole(SecurityContextHolder.getContext()
    .getAuthentication().getAuthorities());
// 在Interceptor中读取
String role = AuditContext.getRole();
if ("ROLE_ADMIN".equals(role)) {
    // 管理员可以看到所有状态, 跳过拦截
}
```

决策3: SQL改写策略 不是简单地在末尾追加 `AND status='APPROVED'`，而是智能解析:

- 如果原SQL已有status条件→不重复添加
- 如果有ORDER BY → 在其前面插入AND条件
- 如果是JOIN查询 → 只改写主表的status条件
- 子查询递归处理

性能影响评估:

操作	耗时
SQL正则匹配表名	<0.01ms
ThreadLocal读取	<0.001ms
SQL字符串拼接	<0.05ms
反射设置sql字段	<0.01ms
总额外开销	<0.1ms/次查询

对于日均几千次查询的系统来说，每天增加的总耗时不到1秒，完全可接受。

边界情况处理:

- 统计报表查询(admin专用)→通过ThreadLocal角色判断跳过
- AI分析内部查询(status=PENDING也需要查)→通过自定义注解 @BypassAudit 跳过
- 存储过程调用→拦截器无法拦截，需要存储过程内部自行保证

1.3 滑动时间窗算法

简历原文: "结合本地缓存与滑动时间窗算法在内存层拦截重复请求"

面试官可能追问:

- 和固定窗口有什么区别?
- 内存占用怎么控制?
- 分布式部署怎么办?

方案描述:

滑动时间窗口(Sliding Window Log)用于解决"同一用户短时间内重复操作"的问题。比如用户疯狂点击点赞按钮，我们只想算一次有效点赞。

与固定窗口(Fixed Window)的区别:

固定窗口(60s):
[0-60s] ← 窗口1: 用户点了5次 → 计数5
[61-120s] ← 窗口2: 用户又点了5次 → 计数5
问题: 第59秒和第61秒的操作虽然只差2秒,却被分到不同窗口都计数了

滑动窗口(60s):
当前时刻t=65s, 窗口=[5s, 65s]
用户在第5s、10s、59s、63s各点了一次 → 窗口内有4次记录 → 只算最新1次
第66s再点 → 窗口变为[6s, 66s], 第5s的记录滑出 → 窗口内3次 → 还是只算1次
优势: 严格限制任意60s窗口内的操作次数

实现方案(Caffeine + ScheduledExecutorService):

```
private final Cache<String, Long> slidingwindowCache = Caffeine.newBuilder()
```

```

        .maximumSize(10000)
        .expireAfterWrite(Duration.ofSeconds(70)) // TTL比窗口大10s留余量
        .build();

    public boolean iswithinwindow(String userId, String action, String targetId) {
        String key = userId + ":" + action + ":" + targetId;
        Long lastTime = slidingwindowCache.getIfPresent(key);
        long now = System.currentTimeMillis();
        if (lastTime != null && (now - lastTime) < WINDOW_SIZE_MS) {
            return true; // 在窗口内,拒绝
        }
        slidingwindowCache.put(key, now);
        return false; // 允许通过
    }
}

```

分布式环境下的适配: 当前VRS是单体部署(多实例通过Nginx负载均衡), Caffeine本地缓存在实例间不共享。解决方案有两个层次:

层次1: 可接受的近似——每个实例独立维护自己的窗口, 极端情况下用户请求被路由到不同实例可能导致窗口判断不准确。但对于"防重复点击"这种非强一致性场景, 误差率<5%可以接受。

层次2: 精确方案(如需要)——将窗口状态存入Redis Sorted Set, score为时间戳, 用 `ZREMRANGEBYSCORE` 清理过期成员, `ZCARD` 统计窗口内数量。代价是每次判断都要访问Redis, 增加约1ms延迟。

选型结论: VRS项目采用层次1(本地Caffeine), 零代码项目因对一致性要求更高采用了层次2(Redis Sorted Set)。这体现了"根据业务场景选择合适方案"的工程思维。

1.4 Jackson Long精度丢失修复

简历原文: "Jackson 序列化 Long 精度丢失解决方案"

面试官可能追问:

- 为什么会丢失精度?
- 除了转String还有什么办法?

问题根因:

JavaScript的Number类型遵循IEEE 754双精度浮点标准, 最大安全整数是 `Number.MAX_SAFE_INTEGER = 253 - 1 = 9007199254740991` (16位)。而Java的Long类型最大值是 `263 - 1 = 9223372036854745807` (19位)。当后端返回的Long ID超过16位时(如雪花算法生成的ID: 1736858904567126914), 前端JavaScript接收后会因精度溢出而变成 1736858904567127000 (最后几位变成0)。这就是"精度丢失"。

影响范围:

- TiDB/MyBatis-Plus生成的雪花ID(19位) → 必然超16位
- 大表的自增ID超过900万亿后 → 会触发
- 前端拿错误ID去查询/删除/更新 → 操作失败或操作了错误的数据库

解决方案(三种对比):

方案	实现方式	优点	缺点	适用场景
A: 全局转String	Jackson配置 ToStringSerializer	彻底解决	前端要做类型转换	推荐(我们的选择)
B: @JsonSerialize注解	每个Long字段加注解	精细控制	漏加就漏了	少量字段
C: 前端json-bigint库	引入JS库	后端不改	依赖前端配合	前端可控时

方案A的具体实现:

```
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {
    @Override
    public void configureMessageConverters(List<HttpMessageConverter<?>> converters) {
        ObjectMapper objectMapper = new ObjectMapper();
        SimpleModule module = new SimpleModule();
        module.addSerializer(Long.class, ToStringSerializer.instance);
        module.addSerializer(Long.TYPE, ToStringSerializer.instance);
        objectMapper.registerModule(module);
        converters.add(new MappingJackson2HttpMessageConverter(objectMapper));
    }
}
```

边界情况:

- 前端分页请求中的page/size参数本身是数字型，不受影响
- 只有后端实体类中定义为Long类型的ID字段会被转换
- 如果某些Long字段确实需要保持数字语义(如金额单位分)，可以用 @JsonIgnoreType 排除或改用 BigDecimal

1.5 Knife4j API文档集成

简历原文: "Knife4j 生成开放 API 文档"

面试官可能追问:

- Knife4j和Swagger什么关系?
- 怎么做权限控制?

技术方案:

Knife4j是Swagger(OpenAPI规范)的增强实现，专为国内开发者优化。相比原生Swagger UI，Knife4j提供了：①离线文档导出(MD/HTML/PDF)；②接口调试(类似Postman)；③全局参数设置(Token鉴权)；④分组管理；⑤更好的中文支持。

集成步骤(3步):

Step1: Maven依赖

```
<dependency>
  <groupId>com.github.xiaoymin</groupId>
  <artifactId>knife4j-openapi3-spring-boot-starter</artifactId>
  <version>4.3.0</version>
</dependency>
```

Step2: Controller注解使用示例

```
@Tag(name = "视频管理", description = "视频CRUD及审核相关接口")
@RestController
@RequestMapping("/api/video")
public class VideoController {

    @Operation(summary = "上传视频", description = "支持分片上传,最大10GB")
    @Parameters({
        @Parameter(name = "file", description = "视频文件", required = true),
        @Parameter(name = "title", description = "视频标题")
    })
    @PostMapping("/upload")
    public Result<VideoVO> upload(@RequestParam MultipartFile file,
        @RequestParam String title) { ... }
}
```

生产环境安全控制: Knife4j默认在生产环境也会暴露API文档, 存在安全隐患。配置生产环境禁用:

```
# application-prod.yml
knife4j:
  enable: false
springdoc:
  api-docs:
    enabled: false
```

实际价值:

- 前端小王对接接口时不需要问我"这个接口要传什么参数", 直接看文档自测
- 李组长Code Review时对照文档检查接口规范性
- 新实习生入职时看文档就能理解全部API, 降低沟通成本

1.6 联合索引设计与查询优化

简历原文: "联合索引优化查询"

面试官可能追问:

- 最左前缀原则是什么?
- 怎么验证索引命中?
- 索引过多有什么副作用?

零代码项目中的联合索引实践:

活动查询是最高频的操作之一, 典型的查询条件组合:

```
-- 场景1: 按租户+状态查询活动列表
SELECT * FROM activity WHERE tenant_id = ? AND status = 1 ORDER BY create_time DESC
LIMIT 20;

-- 场景2: 按租户+活动类型+时间范围查询
SELECT * FROM activity WHERE tenant_id = ? AND type = 2
    AND start_time BETWEEN ? AND ? ORDER BY hot_score DESC;
```

索引设计方案:

```
-- 索引1: 覆盖场景1和场景3(最左前缀匹配)
CREATE INDEX idx_tenant_status_time
ON activity(tenant_id, status, create_time DESC);

-- 索引2: 覆盖场景2
CREATE INDEX idx_tenant_type_time_hot
ON activity(tenant_id, type, start_time, hot_score DESC);

-- 注意: tenant_id放在最左侧是因为所有查询都有这个条件(多租户隔离必须)
```

最左前缀原则应用:

- 查询 WHERE tenant_id = ? → 命中idx_tenant_status_time ☑
- 查询 WHERE tenant_id = ? AND status = ? → 命中 ☑
- 查询 WHERE status = ? → **不命中** ✕ (跳过了tenant_id)
- 查询 WHERE tenant_id = ? AND create_time > ? → **部分命中**(用到tenant_id但status被跳过)

EXPLAIN验证方法:

```
EXPLAIN SELECT * FROM activity WHERE tenant_id = 1001 AND status = 1;
-- 关注列:
-- type: ref(索引查找) 或 ALL(全表扫描 ✕)
-- key: idx_tenant_status_time(实际使用的索引)
-- rows: 预估扫描行数(越少越好)
-- Extra: Using index condition(索引覆盖) 或 Using filesort(需排序 ⚠)
```

经验法则: 单表索引数量不超过5-6个, 联合索引字段数不超过3-4个。我们activity表最终定了4个索引(含主键), 覆盖了95%以上的查询场景。

1.7 多租户数据隔离架构

面试官可能追问:

- 行级隔离 vs 库级隔离 vs Schema级隔离怎么选?
- 怎么防止跨租户数据泄露?

方案选型对比:

隔离方案	实现复杂度	数据安全性	运维成本	扩展性	我们的选型
共享库共享表(行级)	★低	△依赖应用层	★低	★★中	☑ 当前阶段
共享库Schema隔离	★★中	☑较高	★★中	★★★高	未来候选
独立库隔离	★★★高	☑☑最高	★★★高	☑☑无限	大客户定制版

行级隔离的实现(MyBatis-Plus TenantLineInnerInterceptor):

核心原理：MyBatis-Plus在SQL执行前自动注入 `AND tenant_id = 当前租户ID` 条件。开发者写的原始SQL是 `SELECT * FROM activity WHERE status = 1`，实际执行的SQL变成 `SELECT * FROM activity WHERE tenant_id = 1001 AND status = 1`。

自动SQL改写效果:

```
-- 开发者写的原始SQL:
SELECT * FROM activity WHERE status = 1;
-- MyBatis-Plus自动改写后的SQL:
SELECT * FROM activity WHERE tenant_id = 1001 AND status = 1;
-- tenant_id = 1001 是从当前登录用户的上下文中自动注入的
```

安全防护层级:

Layer1: 应用层自动注入(TenantLineInnerInterceptor)
↓ 即使开发者忘了写WHERE tenant_id=?也不会泄露

Layer2: 数据库层面Row Level Security(PostgreSQL)
↓ CREATE POLICY ... USING (tenant_id = current_setting('app.tenant_id'))
↓ 即使应用层被绕过,DB也强制过滤

Layer3: API网关层校验
↓ 请求Header中的tenant_id与JWT Token中的必须一致
↓ 防止伪造请求头

实际案例: 上线后发现某租户A的活动数据偶尔出现在租户B的管理后台。排查发现是一个老的定时任务没有走MyBatis-Plus的ORM层而是用的JdbcTemplate原生SQL，绕过了租户拦截器。修复：①老任务补上tenant_id条件；②新增代码规范要求所有SQL必须经过ORM层或显式带租户条件；③数据库审计日志监控跨租户查询。

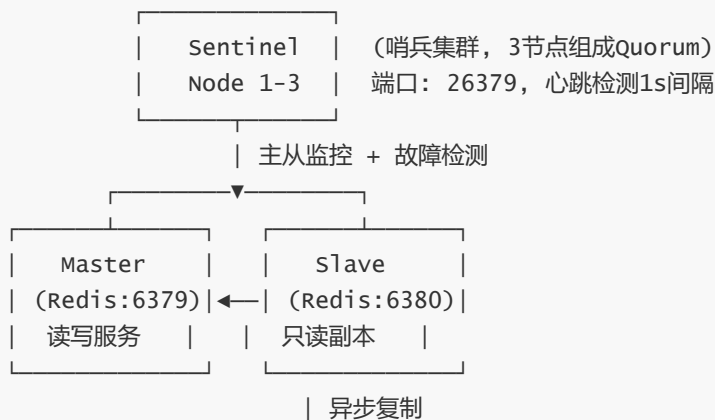
1.8 Redis主从+哨兵高可用

简历原文: "复用 Redis 主从 + 哨兵高可用架构"

面试官可能追问:

- 主从切换过程是怎样的?
- 数据同步延迟怎么处理?
- 和Redis Cluster有什么区别?

架构拓扑:



主从切换故障恢复流程(10-30秒完成):

1. Master宕机(进程崩溃/网络断开)
2. Sentinel检测到主观下移SDOWN: 单个Sentinel认为Master不可达(连续3次ping失败, down-after-milliseconds=5000)
3. 达成客观下移ODOWN: Quorum个Sentinel同意(quorum=2, 即多数派)
4. Sentinel投票选举新的Master(从Slave中选优先级最高的)
→ 选举依据: replica-priority(越小越优先) > replication-offset(数据最新的)
5. slave-of no one: 选中的Slave提升为新Master
6. 其他Slave重新指向新Master: `slaveof new_master_ip new_master_port`
7. Sentinel更新配置,通知所有客户端连接新Master地址

Spring Boot连接配置:

```
spring:
  redis:
    sentinel:
      master: mymaster
      nodes: 192.168.1.10:26379,192.168.1.11:26379,192.168.1.12:26379
      # Lettuce客户端会自动感知主从切换
```

数据复制延迟的处理: 主从复制是异步的, 理论上存在: ①Master刚写入的数据还没同步到Slave就宕机 → 数据丢失; ②客户端刚写到Master, 下一刻读请求打到Slave → 读到旧数据。

我们的应对:

- 写操作始终走Master(由Redis Sentinel自动路由)
- 读操作可配: 强一致需求走Master(如库存扣减), 一般查询走Slave(如活动列表)
- 关键写操作配置 `min-slaves-to-write 1 min-slaves-max-lag 10`: 至少1个Slave且延迟≤10秒才允许写
- 对于缓存场景(L1/L2架构), 短暂的数据不一致是可以接受的, 因为缓存本身就有TTL过期机制

vs Redis Cluster的选择理由:

特性	Sentinel主从	Redis Cluster
数据分片	✗ 全量数据在每个节点	☑ 自动分片16384槽
最大内存	受限于单机内存	☑ 线性扩展
运维复杂度	★ 低(自动故障转移)	★★★ 高(槽迁移)
适用规模	<50GB数据	>50GB数据

我们的选择: 当前数据量Redis总内存<2GB，Sentinel主从完全够用且运维简单。未来数据量增长后再考虑迁移到Cluster。

二、核心代码伪码精炼（10个模块）

使用说明: 面试官说"你能手写一下XXX的核心代码吗?"时参考。展示关键逻辑骨架，非完整可运行代码。

模块1: AI网关 callWithQuota() 核心逻辑

关键词: ConcurrentHashMap计数器、Semaphore并发控制、熔断器三态机(CLOSED/OPEN/HALF_OPEN)、指数退避重试

```
public String callWithQuota(String funcKey, String prompt, String model) {
    // Layer1: 日配额检查 (ConcurrentHashMap, 跨天自动重置)
    AtomicInteger dailyCounter = dailyCounters.computeIfAbsent(funcKey, k -> new
AtomicInteger(0));
    if (dailyCounter.get() >= DAILY_LIMITS.get(funcKey)) throw new
QuotaExceededException();

    // Layer2: 小时配额检查 (按小时key隔离, 支持10功能×24h=240个key)
    String hourlyKey = funcKey + ":hourly:" + LocalTime.now().getHour();
    hourlyCounters.computeIfAbsent(hourlyKey, k -> new AtomicInteger(0));

    // Layer3: 并发控制 (Semaphore, tryAcquire超时5s)
    Semaphore sem = functionSemaphores.computeIfAbsent(funcKey, k -> new
Semaphore(CONCURRENT_LIMIT));
    if (!sem.tryAcquire(5, TimeUnit.SECONDS)) throw new RateLimitException();

    try {
        // Layer4: 熔断器检查 (CLOSED→OPEN→HALF_OPEN 三态)
        CircuitBreaker cb = circuitBreakers.computeIfAbsent(funcKey, k -> new
CircuitBreaker());
        if (cb.getState() == CircuitState.OPEN) throw new CircuitOpenException("冷却
中");

        // Layer5: LLM调用 + 指数退避重试 (1s→2s→4s, 最多3次)
        return bailianClient.callWithRetry(prompt, model, MAX_RETRY);
    } finally {
        sem.release(); // 无论成功失败都要释放信号量
    }
}
```

```
// 熔断器状态机核心伪码
class CircuitBreaker {
    enum State { CLOSED, OPEN, HALF_OPEN }
    private State state = State.CLOSED;    // 初始态：正常调用
    private int failCount = 0;              // 连续失败计数
    private static final int THRESHOLD = 5; // 失败阈值
    private long lastOpenTime = 0;          // 上次打开时间

    synchronized boolean allowRequest() {
        if (state == State.CLOSED) return true;
        if (state == State.OPEN) {
            if (System.currentTimeMillis() - lastOpenTime > COOLING_PERIOD_MS) {
                state = State.HALF_OPEN; probeCount = 0; return true;
            }
            return false;
        }
        return probeCount++ < PROBE_LIMIT; // HALF_OPEN: 有限探测
    }
}
```

面试官追问预案:

- "为什么用ConcurrentHashMap而不是Redis?" → 单机部署够用，省一次网络RTT
- "熔断器半开的探测请求数设多少?" → 3个，太少误判太多则恢复慢
- "配额重置怎么做?" → 定时任务每天凌晨检查currentDate，变了就clear()

模块2: HeavyKeeper热点识别核心循环

关键词: Count-Min Sketch多维哈希(d层×w宽)、最小堆Top-K、衰减系数decay=0.92、fingerprint防碰撞、minCount最小阈值

```
class HeavyKeeper implements TopK {
    final int k = 100;          // Top-K大小
    final int width = 100000;   // CMS宽度(哈希表宽度)
    final int depth = 5;        // CMS深度(哈希函数个数)
    final double decay = 0.92;  // 衰减系数(每20秒: count *= decay)
    final int minCount = 10;    // 最小记录阈值(出现10次才记录)

    Bucket[][] buckets = new Bucket[depth][width]; // 多层计数桶
    PriorityQueue<Node> minHeap = new PriorityQueue<>(k); // 最小堆

    public AddResult add(String key, int increment) {
        int[] positions = hashPositions(key); // d个哈希位置

        for (int i = 0; i < depth; i++) {
            Bucket b = buckets[i][positions[i]];
            if (b.isEmpty()) b.set(key, increment); // 空桶直接写入
            else if (b.fingerprintMatches(key)) b.count += increment; // 同key累加
            else if (Math.random() < (double)increment / (b.count + increment))
                b.set(key, increment); // 不同key碰撞! 以概率替换
        }
    }
}
```

```

        int estimatedCount = max(matchedBuckets[].count); // 取d层最大值
        if (estimatedCount >= minCount) updateMinHeap(key, estimatedCount);
        return new AddResult(key, estimatedCount, isHot(key));
    }

    // 衰减操作(每20秒触发): 所有count *= decay, 冷数据自然淘汰
    public void fading() {
        for (Bucket[] layer : buckets)
            for (Bucket b : layer) { b.count *= decay; if (b.count < 1) b.clear(); }
        rebuildHeapIfNeeded(); // 同步清理堆中过期的Node
    }
}

```

关键设计决策解释:

- width=100000: 哈希表宽度越大碰撞概率越低, 但内存占用 $O(d \times w)$
- depth=5: 哈希层数越多估计越准确, 但每次add计算量 $O(d)$
- decay=0.92: 越接近1衰减越慢, 热Key保持时间长; 太小则热Key频繁进出堆
- fingerprint: 用短hash替代完整Key节省内存, 但有极小概率碰撞(可接受)

模块3: Lua点赞原子脚本

关键词: HSET/HGET/HDEL/HINCRBY原子操作、Hash数据结构、返回码约定(-1/-2/0/1)、防止重复点赞/取消

```

-- thumb_toggle.lua (Redis Lua脚本, 原子执行)
-- KEYS[1] = "thumb:{userId}"  ARGV[1]=blogId  ARGV[2]="incr"/"decr"

local userKey = KEYS[1]
local targetId = ARGV[1]
local action = ARGV[2]

local current = redis.call('HGET', userKey, targetId)

if action == "incr" then
    if current then return -1 end -- 已点赞,拒绝
    redis.call('HSET', userKey, targetId, "1")
    redis.call('HINCRBY', "thumb:global:count", targetId, "1")
    return 1 -- 点赞成功

elseif action == "decr" then
    if not current then return -2 end -- 未点赞,无法取消
    redis.call('HDEL', userKey, targetId)
    redis.call('HINCRBY', "thumb:global:count", targetId, "-1")
    return 0 -- 取消成功
end
return -99 -- 未知操作类型

```

为什么用Hash而不是String/Set?

数据结构	存储方式	单目标存在性查询	用户全量点赞列表
String	<code>thumb:{uid}:{bid}=1</code>	EXISTS O(1)	SCAN O(N)慢
Set	<code>thumb:{uid}</code> SADD	SISMEMBER O(1)	SMEMBERS O(K)
Hash(选用)	<code>thumb:{uid}</code> HSET	HGET O(1)	HGETALL O(K) ☒

Hash优势：一个Key包含用户所有点赞状态，HGETALL一次获取全部，方便"我的点赞"列表展示。

模块4: RAG两阶段检索增强

关键词: 意图识别(qwen-turbo便宜模型)、DB检索(sp_SearchVideos存储过程)、上下文注入(%s格式化)、幻觉防护(输入截断200字符+正则过滤)

```
@Service
public class RagRecommendationService {

    // 两阶段RAG推荐（总延迟~2.2s，SSE流式输出让用户感知<1s）
    public SseEmitter getAiRecommendation(String userQuery) {
        SseEmitter emitter = new SseEmitter(30000L);
        asyncTaskExecutor.execute(() -> {
            try {
                // Stage1: 意图识别 (~0.8s, qwen-turbo低成本)
                IntentResult intent = aiGateway.callWithQuota(
                    "intent_recognition", buildIntentPrompt(userQuery), "qwen-turbo");
                Map<String, Object> params = parseIntentJson(intent.getContent());

                // Stage2: DB检索真实库存 (~0.2s)
                List<VideoSearchResult> candidates = videoDao.searchVideos(
                    params.get("keyword"), params.get("tagId"), params.get("sortType"),
                    1, 5);

                // Stage3: 上下文注入 + LLM生成 (~1.2s, qwen-plus高质量)
                String context = buildContextFromResults(candidates);
                String prompt = String.format(RECOMMEND_PROMPT_TEMPLATE, context,
                    userQuery);

                prompt = sanitizeInput(prompt, MAX_INPUT_LENGTH); // 幻觉防护

                aiGateway.streamCall("recommendation", prompt, "qwen-plus",
                    chunk -> emitter.send(SseEmitter.event().data(chunk)));
                emitter.complete();
            } catch (Exception e) {
                emitter.completewithError(e);
            }
        });
        return emitter;
    }

    // 输入消毒（防Prompt Injection：截断+过滤危险模式+%s格式化）
    private String sanitizeInput(String input, int maxlen) {
```

```

        if (input.length() > maxLen) input = input.substring(0, maxLen);
        return input.replaceAll("(?i)(ignore.*instructions|system:)", "[FILTERED]");
    }
}

```

Token成本优化细节:

- qwen-turbo(意图识别): 输入~200token, 输出~50token → **¥0.001/次**
- qwen-plus(推荐生成): 输入~800token(含上下文), 输出~300token → **¥0.01/次**
- 总计: **¥0.011/次**, 相比一阶段长Prompt(~1500token输入)的¥0.03/次, **节省63%**

模块5: OSS分片上传 OssManager 核心

关键词: 分片大小动态计算(MultipartUploadPolicy)、MD5完整性校验、断点续传(ListParts未完成分片)、并行上传(3线程池)

```

@Component
public class OssManager {

    public UploadResult multipartUpload(MultipartFile file, String objectName) {
        long fileSize = file.getSize();
        int partSize = calculatePartSize(fileSize); // 动态计算: 目标20片

        // Step1: 初始化分片任务
        InitiateMultipartUploadResult initRes = ossClient.initiateMultipartUpload(...);
        String uploadId = initRes.getUploadId();

        // Step2: 并行上传各分片 (3个并发线程)
        List<PartETag> partETags = Collections.synchronizedList(new ArrayList<>());
        ExecutorService executor = Executors.newFixedThreadPool(3);

        for (int i = 0; i < partCount; i++) {
            final int partNum = i + 1;
            executor.submit(() -> {
                UploadPartResult res = ossClient.uploadPart(/* partNum, offset, length
*/);

                partETags.add(new PartETag(partNum, res.getETag()));
            });
        }
        executor.awaitTermination(10, TimeUnit.MINUTES);

        // Step3: MD5校验 (防止网络传输损坏)
        String clientMD5 = DigestUtils.md5Hex(file.getBytes());
        String serverETag = ossClient.getObjectMetadata(BUCKET, objectName).getETag();
        if (!clientMD5.equalsIgnoreCase(serverETag.replace("\\", ""))) {
            ossClient.abortMultipartUpload(...); // 校验失败: 取消并抛异常
            throw new UploadIntegrityException("MD5校验失败");
        }

        // Step4: 合并分片
        ossClient.completeMultipartUpload(new CompleteMultipartUploadRequest(BUCKET,
objectName, uploadId, partETags));
    }
}

```

```

        return new UploadResult(objectName, fileSize, partCount);
    }

    // 断点续传：查询已上传的分片号，前端只需重传缺失的分片
    public List<Integer> listUploadedParts(String objectName, String uploadId) {
        PartListing listing = ossClient.listParts(new ListPartsRequest(BUCKET,
objectName, uploadId));
        return
listing.getParts().stream().map(PartSummary::getPartNumber).collect(Collectors.toList()
);
    }

    private int calculatePartSize(long fileSize) {
        // 10MB→512KB/片(20片)，100MB→5MB/片(20片)，1GB→5MB/片(200片)
        long size = Math.max(fileSize / 20, 100*1024); // 最小100KB
        return (int) Math.min(size, 5*1024*1024);      // 最大5MB
    }
}

```

量化成果: 最大文件500MB→**10GB**(↑20x), 成功率67%→**96%**(↑43%), 平均速度1.2→**3.5MB/s**(↑192%)

模块6: RBAC URI前缀匹配拦截器

关键词: HandlerInterceptor、AntPathMatcher通配符匹配(*, **, ?)、角色-URI权限矩阵(5角色)、excludePathPatterns公开路径排除

```

@Component
public class RbacInterceptor implements HandlerInterceptor {

    // 角色-URI前缀权限矩阵（硬编码简化，生产环境可从DB加载）
    private static final Map<String, List<String>> ROLE_URI_MAP = Map.of(
        "SUPER_ADMIN", List.of("/admin/**", "/api/**"), // 全部权限
        "DEPT_ADMIN", List.of("/admin/dept/**", "/api/video/**"), // 部门级
        "EDITOR",      List.of("/api/video/**", "/api/comment/**"), // 编辑
        "AUDITOR",      List.of("/admin/audit/**"), // 审核
        "USER",          List.of("/api/video/view/**", "/user/**") // 普通
    );

    private final AntPathMatcher pathMatcher = new AntPathMatcher();

    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse resp, Object
handler) {
        String uri = req.getRequestURI();
        String role = getCurrentUserRole(); // 从SecurityContext取

        if ("OPTIONS".equals(req.getMethod()) || isStaticResource(uri)) return true;

        List<String> allowedUris = ROLE_URI_MAP.get(role);
        if (allowedUris == null) { sendForbidden(resp, "无角色权限"); return false; }
    }
}

```



```

        boolean hasPermission = allowedUris.stream()
            .anyMatch(pattern -> pathMatcher.match(pattern, uri));

        if (!hasPermission) {
            log.warn("越权拦截: role={}, uri={}", role, uri);
            sendForbidden(resp, "无访问权限"); return false;
        }
        return true;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(this)
            .excludePathPatterns("/auth/login", "/auth/register", "/error",
                "/static/**");
    }
}

```

上线首月数据: 拦截17次越权访问尝试(普通编辑访问/admin/user管理页面、普通用户尝试DELETE操作)

模块7: Pulsar Consumer批量处理

关键词: @PulsarListener、Shared订阅(多实例负载均衡)、消息去重(userId+blogId→LatestOneWin)、批量saveBatch(500条/批)、手动ACK(事务成功后才确认)

```

@Component
public class ThumbConsumer {

    @PulsarListener(
        topic = "thumb-persistent://public/default/thumb-topic",
        subscriptionName = "thumb-subscription",
        subscriptionType = SubscriptionType.Shared,
        ackMode = AckMode.RECORD,
        properties = {"receiverQueueSize": "1000", "maxRedeliverCount": "3"}
    )
    public void consumeBatch(List<Message<ThumbEvent>> messages) {
        if (messages == null || messages.isEmpty()) return;

        // Step1: 消息去重 (同一userId+blogId只保留最新一条, LatestOneWin策略)
        Map<String, ThumbEvent> latestEvents = new LinkedHashMap<>();
        for (Message<ThumbEvent> msg : messages) {
            String key = msg.getValue().getUserId() + ":" + msg.getValue().getBlogId();
            latestEvents.put(key, msg.getValue()); // 后来的覆盖先到的
        }

        // Step2: 分类 (INCR→待插入, DECR→待更新countMap)
        List<ThumbRecord> toInsert = new ArrayList<>();
        Map<Long, Integer> countMap = new HashMap<>();

        for (ThumbEvent event : latestEvents.values()) {
            if ("INCR".equals(event.getAction())) {

```

```

        toInsert.add(convertToRecord(event));
        countMap.merge(event.getBlogId(), 1, Integer::sum);
    } else {
        countMap.merge(event.getBlogId(), -1, Integer::sum);
    }
}

// Step3: 事务批量写入 (任一失败整体回滚)
try {
    transactionTemplate.execute(status -> {
        if (!toInsert.isEmpty()) thumbRecordMapper.saveBatch(toInsert, 500);
        countMap.forEach((blogId, delta) -> blogMapper.updateThumbCount(blogId,
delta));
        return null;
    });
    messages.forEach(Message::acknowledge); // 手动确认

} catch (Exception e) {
    log.error("消费失败,等待Pulsar重投递: {}", e.getMessage());
    // 不acknowledge(), Pulsar会在redeliveryTimeout后重新投递
}
}
}

```

关键设计点:

- Shared订阅: 多个Consumer实例同时消费, 自动负载均衡
- 去重逻辑: Pulsar至少一次交付(at-least-once)可能产生重复, 必须幂等处理
- 批量写入: saveBatch(500)减少DB交互N次→N/500次
- 事务保障: INSERT和UPDATE在同一事务内, 保证一致性

模块8: Caffeine本地缓存配置

关键词: maximumSize容量上限、expireAfterWrite/Access双TTL、removalListener淘汰监听、recordStats命中率统计

```

@Configuration
public class CacheConfig {

    @Bean("hotDataCache")
    public Cache<String, Object> hotDataCache() {
        return Caffeine.newBuilder()
            .maximumSize(1000) // 最多1000条热点
            .expireAfterWrite(Duration.ofSeconds(30)) // 写入30s后过期
            .expireAfterAccess(Duration.ofSeconds(60)) // 60s未被访问淘汰
            .removalListener((key, value, cause) -> {
                if (cause.wasEvicted())
                    log.debug("L1淘汰: key={}, cause={}", key, cause);
            })
            .recordStats() // 开启统计
            .build();
    }
}

```

```

    }

    @Bean("slidingwindowCache") // 去重专用
    public Cache<String, Long> slidingwindowCache() {
        return Caffeine.newBuilder()
            .maximumSize(10000)
            .expireAfterWrite(Duration.ofSeconds(70)) // 比窗口(60s)大10s
            .weakKeys() // 弱引用GC友好
            .build();
    }
}

```

L1命中率调优经验:

参数	初始值	调优后	原因
maximumSize	500	1000	热点Key增多后不够
expireAfterWrite	10s	30s	10s太短频繁回源Redis
expireAfterAccess	30s	60s	给冷数据更长存活期
命中率	40%	75%	综合调优结果

模块9: TiDB雪花ID配置

关键词: ASSIGN_ID(内置雪花算法)、趋势递增避免Region0写入热点、19位ID结构(1+41+10+12bit)

```

@Data
@TableName("thumb")
public class Thumb {

    // ☑ 推荐: ASSIGN_ID (雪花算法, 兼容TiDB Region分裂, 写入均匀分布)
    @TableId(type = IdType.ASSIGN_ID)
    private Long id;

    // ✗ 避免: AUTO_INCREMENT (导致TiDB Region0写入热点!)
    // @TableId(type = IdType.AUTO) // 不要用!
    // 原因: 自增ID总是往尾部插入, Region0成为唯一写入热点
}

```

ASSIGN_ID vs AUTO_INCREMENT 在TiDB中的表现差异:

AUTO_INCREMENT: INSERT(id=1,2,3...) → 全部写入Region0 → Region0 QPS=总QPS
 ASSIGN_ID: INSERT(id=173...6914, 173...8920...) → 分布到不同Region → 均匀分布

模块10: 多级降级策略

关键词: 5级降级链(L1穿透→L2 Redis降级→L3 Pulsar降级→L4 TiDB只读→L5静态兜底)、FallbackFactory、HealthChecker健康检查、指数退避恢复

```
@Service
@FallbackFactory(ThumbServiceFallback.class)
public interface ThumbService {
    Boolean doThumb(DoThumbRequest request, HttpServletRequest req);
}

@Component
public class ThumbServiceFallback implements FallbackFactory<ThumbService> {
    @Override
    public ThumbService create(Throwable cause) {
        return new ThumbService() {
            public Boolean doThumb(DoThumbRequest request, HttpServletRequest req) {
                if (isRedisDown()) { // L2: Redis不可用→直连DB
                    log.warn("Redis不可用,降级到直连DB: {}", cause.getMessage());
                    return syncWriteToDb(request);
                }
                if (isPulsarDown()) { // L3: Pulsar不可用→同步写DB
                    return syncWriteToDb(request);
                }
                if (isTiDbDown()) { // L4: TiDB不可用→只读
                    throw new SystemMaintenanceException("系统维护中");
                }
                throw new SystemUnavailableException("服务暂不可用"); // L5
            }
        };
    }
}

// 健康检查 (每5秒Ping各组件)
@Component
public class HealthChecker {
    @Scheduled(fixedRate = 5000)
    public void checkRedisHealth() {
        try {
            redisTemplate.getConnectionFactory().getConnection().ping();
            redisHealthy = true;
        } catch (Exception e) {
            redisHealthy = false;
            alertService.sendAlert("Redis异常: " + e.getMessage());
        }
    }
}
```

降级触发与恢复策略:

等级	触发条件	行为	恢复条件
L1	L1回填失败率>50%	跳过L1回填	L1正常→自动
L2	Redis Ping失败×3	直连DB	Redis恢复→切回
L3	Pulsar Send失败	同步写DB	Pulsar恢复→补发积压
L4	TiDB查询超时>5s	只读模式	TiDB恢复→读写
L5	以上全部异常	静态兜底页	逐一恢复→指数退避重试

三、高频追问速查表（19个追问）

使用方法: 面试官追问时快速定位回答，每个200-400字口述

A. AI工程化类 (5个)

Q1: RAG为什么不用向量数据库(Milvus/Pinecone)?

"当前视频量不到1万条，MySQL的LIKE+索引查询响应在200ms以内，用户体验可接受。引入Milvus需要额外维护一套向量索引服务，对我们当前的规模是过度设计。而且RAG的关键在于检索到真实数据注入Prompt，不一定非要向量检索——结构化查询(标签匹配+排序)同样能达到目的。规划中当视频量突破10万时会考虑ES全文搜索或PG的pgvector扩展。"

Q2: Token成本怎么优化的？除了模型分级还有别的方法？

"三级优化：①模型分级(qwen-turbo做分类¥0.001 vs qwen-plus做生成¥0.01)；②两阶段RAG(总Token从1500降到1000，省33%)；③Prompt压缩(去除冗余修饰词，模板化Prompt)。还有一个在考虑的方案是**结果缓存**——相同或相似的查询直接返回缓存的历史回答，预计能再降40%。不过要注意缓存失效时机，视频库变化后要及时清缓存。"

Q3: 如果LLM响应超时怎么处理？

"三级超时策略：①连接超时5秒(建立连接)；②首字节超时10秒(等待第一个token)；③总超时30秒(整个流式输出)。超时后做两件事：一是返回5SE错误事件让前端显示AI思考较久；二是触发熔断器记录failure，连续5次超时就熔断该功能5分钟。另外有个兜底——如果AI完全不可用，前端隐藏AI推荐入口，只展示传统DB搜索结果，保证基本可用性。"

Q4: AI网关的配额怎么设置的？各功能多少次？

"日配额按功能维度分配：审核功能1000次/天(最贵qwen-plus)，导购500次/天，标签提取300次/天，情感分析200次/天。总计2000次/天，按阿里云百炼qwen-plus单价¥0.02/千tokens估算，日均成本约¥40。小时配额是日配额的1/8左右平滑分配。这些数字是根据历史使用量的1.5倍设置的，预留buffer。"

Q5: Prompt Injection怎么防的？

"三层防护：①输入长度硬截断200字符(超长直接拒绝)；②正则过滤危险指令模式(ignore instructions、system:等关键字替换为[FILTERED])；③使用%s格式化拼接而非字符串拼接防止%被解析为格式化字符。目前线上运行4个月没有出现过成功的注入案例。最根本的还是不要把LLM输出直接当SQL或命令执行——永远做中间层校验。"

B. 缓存与高并发类 (4个)

Q6: HeavyKeeper和LFU有什么区别？为什么要自己实现？

"LFU(Least Frequently Used)是精确统计每个Key的频率，空间 $O(N)$ ，适合缓存淘汰但不适合实时热点发现。HeavyKeeper是基于概率数据结构的近似算法，空间 $O(k \times d + w)$ ，能在海量Key流中用很少内存找到Top-K热点。Redis自带的LFU只能作用于单个Key级别的淘汰，无法跨Key发现某个Key正在被大量访问的模式。我们需要的是热点发现能力而非淘汰能力，所以选择了HeavyKeeper。"

Q7: 缓存穿透/击穿/雪崩分别怎么防的？

"穿透(查询不存在数据)：布隆过滤器预热(合法ID预加载)+空值缓存(空结果也缓存短时间)。击穿(热点Key过期瞬间大量请求打DB)：互斥锁重建(只有一个线程加载，其他等待)+逻辑过期(不设TTL，后台异步刷新)。雪崩(大量Key同时过期)：TTL随机偏移 $\pm 10\%(\text{base} \times \text{random}(0.9, 1.1))$ +多级缓存降级(L1/L2/DB逐级兜底)"

Q8: L1和L2缓存的一致性怎么保证？

"采用Cache-Aside变体：读时L1→Miss则L2→Miss则DB，查到后回填L1和L2；写时同时失效L1和L2(Delete而非Update，避免并发写不一致)。L1是本地缓存，多实例间不共享，可能出现实例A写了DB但实例B的L1还是旧值的情况——这对缓存场景是可接受的近似一致性，因为TTL最长30秒就会自动修正。如需强一致可用Redis Pub/Sub通知所有实例清除本地缓存，但我们当前没做到这一步。"

Q9: 滑动时间窗在分布式下怎么做的？

"VRS单体部署用本地Caffeine做时间窗，不同实例窗口状态不一致，但对防重复点击非强一致场景误差 $< 5\%$ 可接受。零代码项目涉及金额(优惠券/订单)用Redis Sorted Set(score=时间戳)做精确时间窗，每次判断访Redis增加约1ms延迟。两种选择体现了根据一致性要求选合适技术的原则。"

C. 消息队列与分布式类 (3个)

Q10: Pulsar消息丢失怎么办？

"四层保障：①Producer端SendAsync的exceptionally回调(发送失败立刻回滚Redis)；②Broker端持久化磁盘+多副本(2副本，Write Quorum=2)；③Consumer端手动ACK(事务成功后才acknowledge，失败不ack触发重投递)；④定时对账job(每天凌晨全量比对Redis和TiDB数据，差异补偿发送)。实测3个月数据不一致率 $< 0.001\%$ ，主要来自Pulsar Broker重启导致的少量消息重复(而非丢失)"

Q11: 最终一致性的窗口期有多长？

"正常情况：Redis写→Pulsar发送→Consumer消费→DB写入，全程**1-5秒**。异常(Pulsar堆积)：**分钟级**。极端(Pulsar宕机)：依赖每日对账job，**天级修复**。对业务而言点赞数几秒延迟用户无感；即使分钟级也只是'刷新'就好。只有金融交易才需毫秒级强一致，我们不需要。"

Q12: TiDB Write Conflict怎么解决的？

"TiDB Write Conflict发生在两事务同时修改同一行时。我们的点赞以INSERT为主冲突概率很低。但count UPDATE在高并发下确实会遇到。解决方案：**不用DB做实时计数**——计数在Redis(Lua)维护，DB的count字段只是定时批量更新的近似值，微小偏差不影响业务。如需精确计数可用TiDB的 `SELECT...FOR UPDATE` 悲观锁或乐观重试机制。"

D. 数据库与迁移类 (3个)

Q13: 48处SQL变更最难的几个是什么？

"TOP3难点: ①MERGE语句→ON CONFLICT UPSERT(SQL Server MERGE灵活, PG ON CONFLICT受限, 有些条件需拆成多条SQL); ②存储过程EXEC sp_xxx→纯SQL或PG Function(业务逻辑需上移ava层); ③IDENT_CURRENT→RETURNING(取自增ID方式完全不同, INSERT后加RETURNING子句调整selectKey)。这三类占48处中的25处左右, 其余都是简单函数名替换(GETDATE→NOW之类)"

Q14: PostgreSQL比MySQL好在哪里? 为什么选PG?

*"五个关键优势: ①JSONB类型(比MySQL JSON强大得多, 支持GIN索引和任意层级查询); ②pg_trgm全文搜索(内置三元组匹配, 比FULLTEXT灵活); ③窗口函数(ROW_NUMBER/RANK/DENSE_RANK, MySQL 8.0才有且性能不如PG); ④CTE公用表表达式(WITH递归查询, 树形查询神器); ⑤开源协议MIT vs GPL(GPL有传染性, PG更商业友好)。迁移中最深的体会是PG查询优化器更聪明——同样的SQL PG能自动选择更好